Verifying Integrity

If you remember when we first looked hashing functions in an earlier section (`md5()`, `crc32()`, etc.) I mentioned that hashing could also be used to verify data integrity and also sender identity. For these cases we do not need the complexity cost of a password hashing algorithm, because these types of application use shared secrets to "salt" the hash: by incorporating a shared secret key into the source string before hashing we ensure that the only way to reproduce the hash value at the receiving end is by possessing the same secret and original message. The name for this form of message integrity validation is *hash-based message authentication code* ("HMAC") and PHP's core `hash` extension provides some HMAC functions.

`hash_hmac()`

It's time to play spies. I have the secret key (the creation and distribution of such a shared secret is not shown here, but we can imagine it involves pieces of paper that have to be swallowed after reading), and a vital communication to send. By generating an HMAC hash I am able to verify my identity (or at least verify I know the key) and the message contents in one go. I can call `hash_hmac()` with three arguments: first the algorithm I wish to use, then the message, then the shared secret.

```
$secret = 'MySecretKey';
$message = 'The eagle flies at dawn.';
print hash_hmac("sha256", $message, $secret);
```

```
Output:
d41ecc4ad873d37f02e205f408f3d3d2275dc30eab0cee51ea7cb7b4764d9c33
```

The message would then be sent alongside the hash, the receiving end repeats the hash operation (the receiver has their own copy of the secret key, this must never be transmitted for obvious reasons), if the receiver's hash matches the transmitted hash then they know they can trust the message has not been changed and it was signed with the correct key. The message itself has not been encrypted in this scenario, what we're doing here is establishing trust in the sender and the validity of the data.